TAKE-HOME ASSESSMENT · SEPTEMBER 2026

NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE | College of Computing and Data Science

Project Reporting Builder

From raw project notes to snippet cards you can paste into email and chat

Zhang Dingpu

Master of Computing in Applied AI
College of Computing and Data Science
Nanyang Technological University, Singapore

→ chapters · ↓ detail within a chapter

The problem

Weekly status reports are rebuilt by hand, from scattered inputs, for channels that each want a different format.



↓ 2 slides below

THE PROBLEM

Three inputs, three channels, one ambiguity

INPUTS

Scattered

Notes in a doc, numbers in a spreadsheet, a screenshot in a chat thread.

CHANNELS

Each wants its own format

Email renders HTML, chat wants plain text, a slide wants an image.

NUMBERS

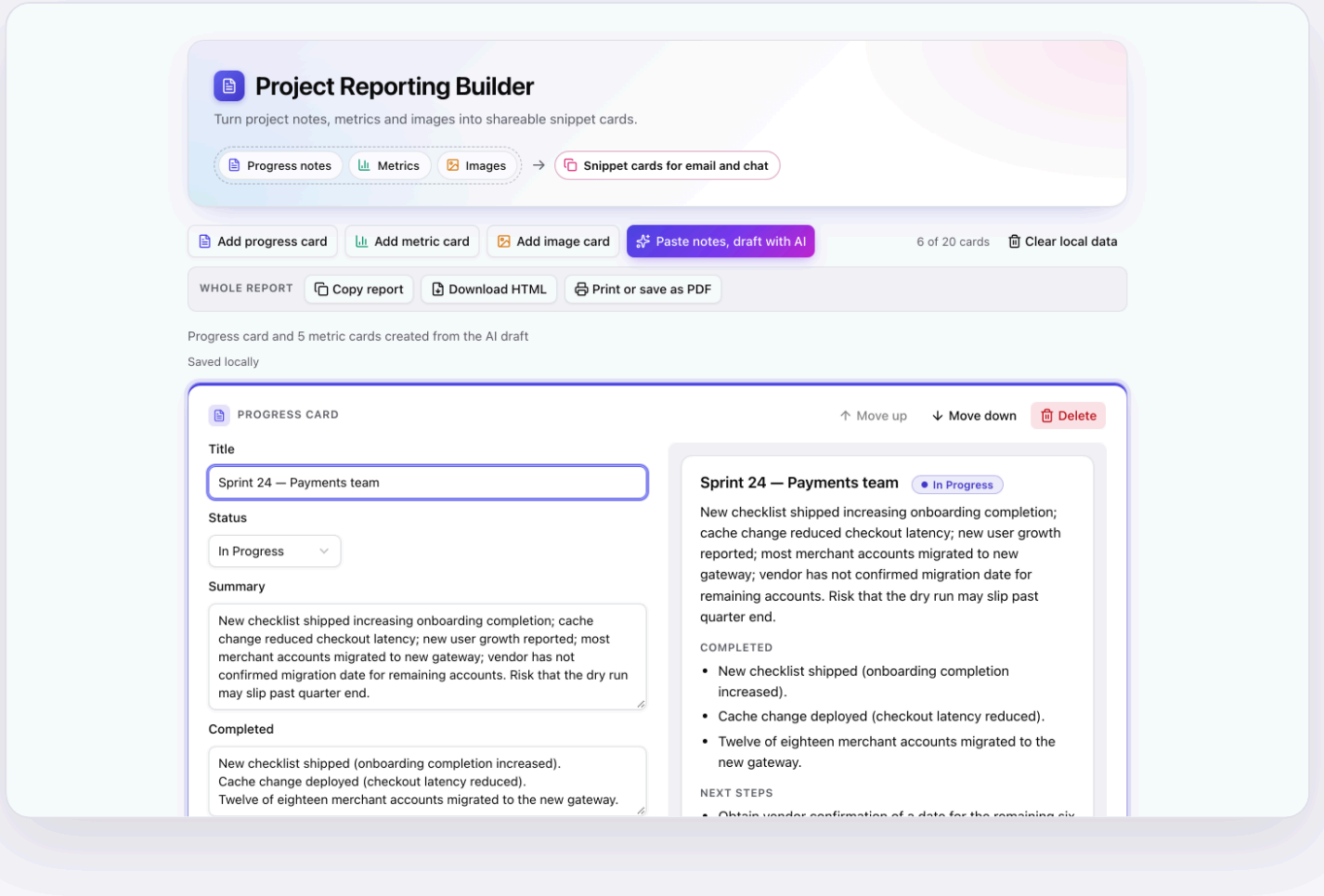
Easy to state wrongly

“+12%” can mean twelve percentage points or a twelve percent change. Different claims.

WHAT IT DOES

Three kinds of input, one kind of output

- Progress notes, metrics and images, in any mix
- AI drafts cards from notes or an image, for review
- Live preview, rendered by the server
- Share one card or the whole report; drafts survive a reload



Live demo

STEP 1

A metric card: 67 → 79

STEP 2

Paste notes or drop an image, draft with AI

STEP 3

Review what the model could not confirm

STEP 4

Copy, PNG, whole-report export

Design decisions

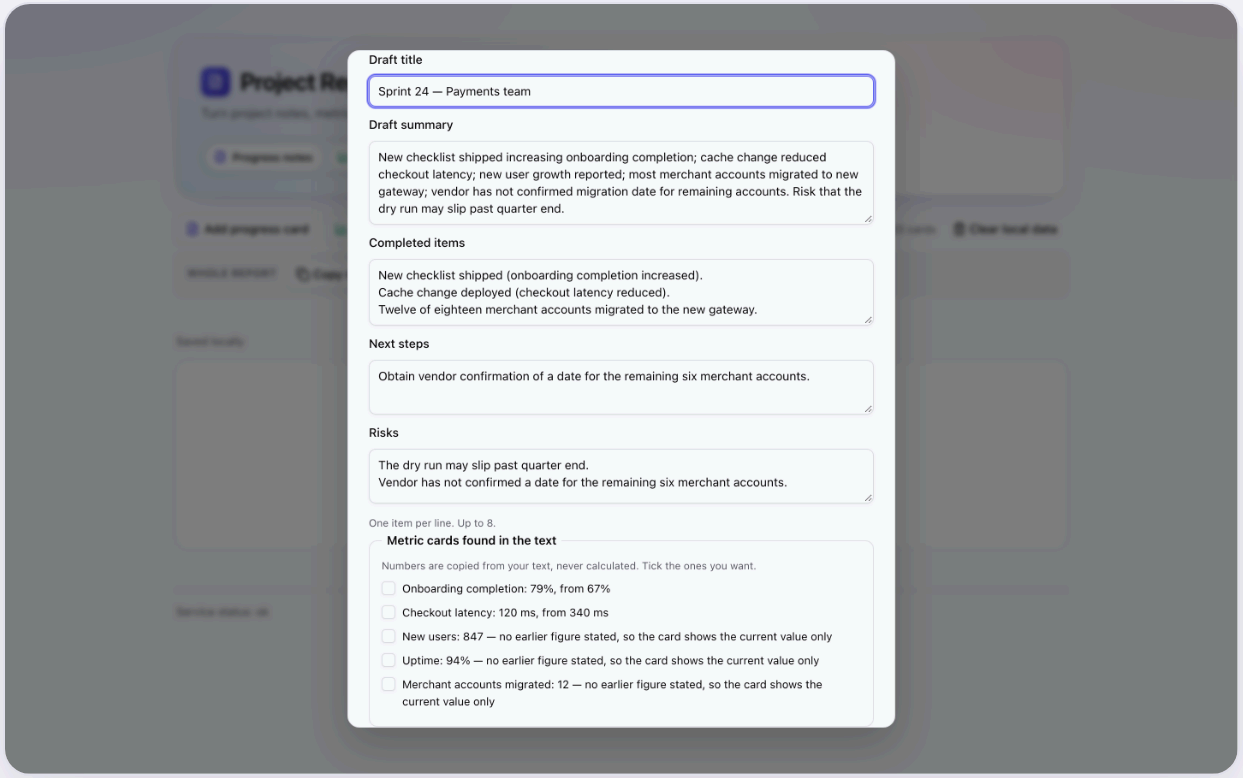
Where the AI stops, how it reads images, the one calculation the tool makes, and why every export matches the preview.



↓ 6 slides below

AI ASSISTANCE

The AI drafts. The person decides.



The model never sets the status

Create stays disabled until the person picks one.

Every draft is flagged for review

With a list of what the model could not confirm.

Metric cards are opt-in

Each proposal starts unticked.

Rules and notes travel apart

Prompt in `instructions`, notes in `input`, strict JSON schema out.

THE BOUNDARY

The model copies numbers. It never derives them.

THE NOTES SAID

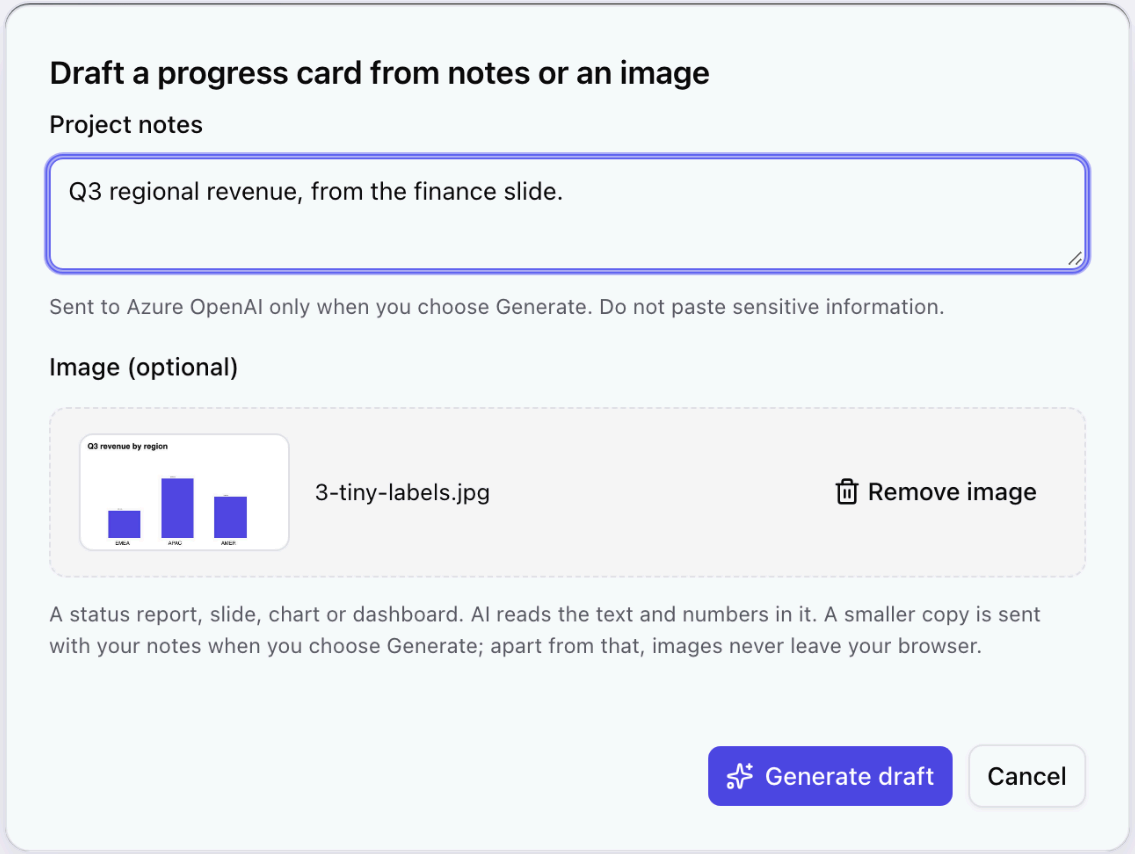
“New users: 847, up 23% from last sprint.”

THE LIVE MODEL WROTE

“…does not provide the previous new-user count; previous is therefore null.”

847 ÷ 1.23 ≈ 688 is a number the notes never state. It appears nowhere in the output, and a browser test asserts that it never does.

A screenshot is often all someone has



Two ways in

An image with the notes drafts a progress card. On an image card, AI drafts its alt text and caption, so a chart survives a text export.

The one exception to “images stay local”

Sent only on request, after the image is shown, as a JPEG of at most 1024 px. Never stored or logged.

Same rules as text

Copy what is visible, never calculate; text inside the image is data, not instructions.

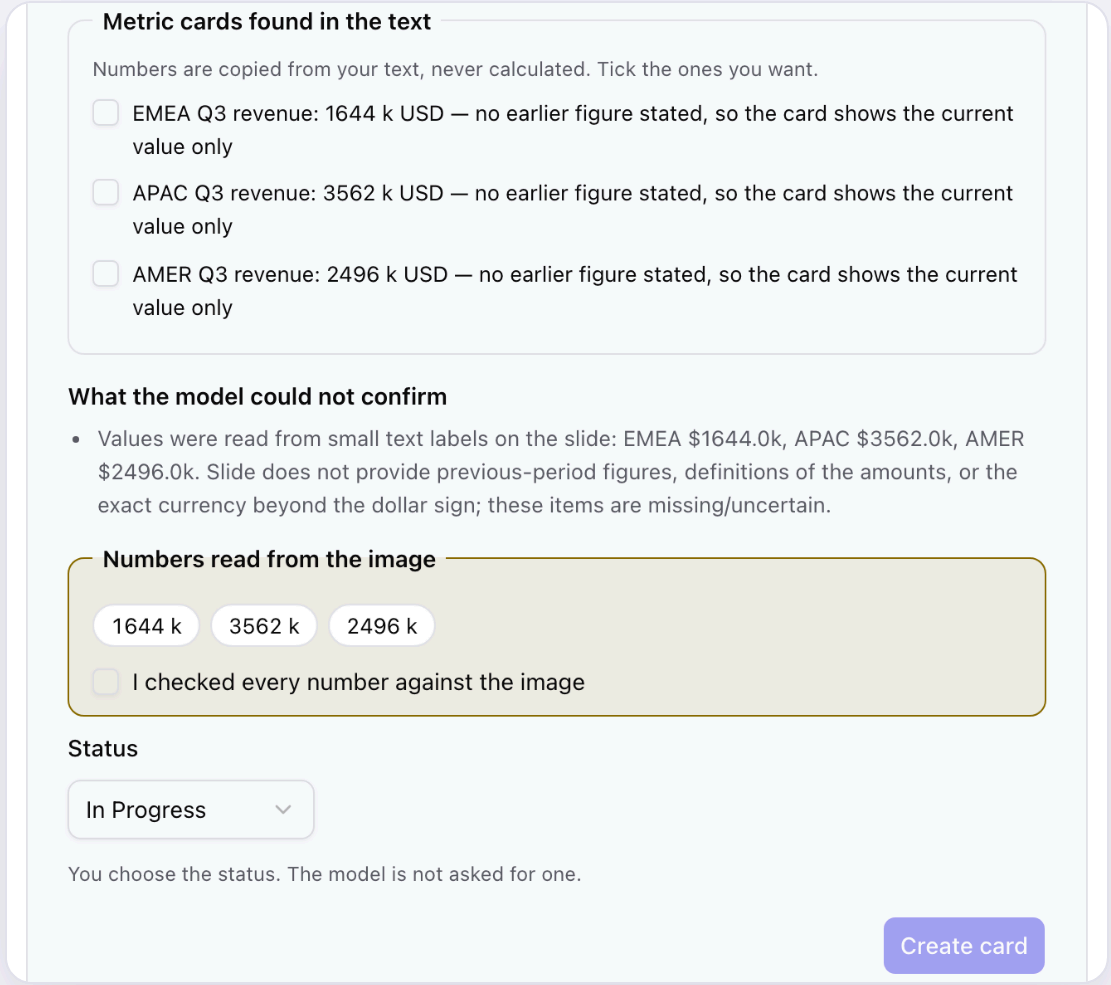
THE FINDING

The model misread small print, and did not say so

LABEL ON THE CHART	WHAT THE LIVE MODEL WROTE
$1644.0k	$1464.0k, $1464.0k, $1044.0k
$2466.0k	$2496.0k, today, while making this slide

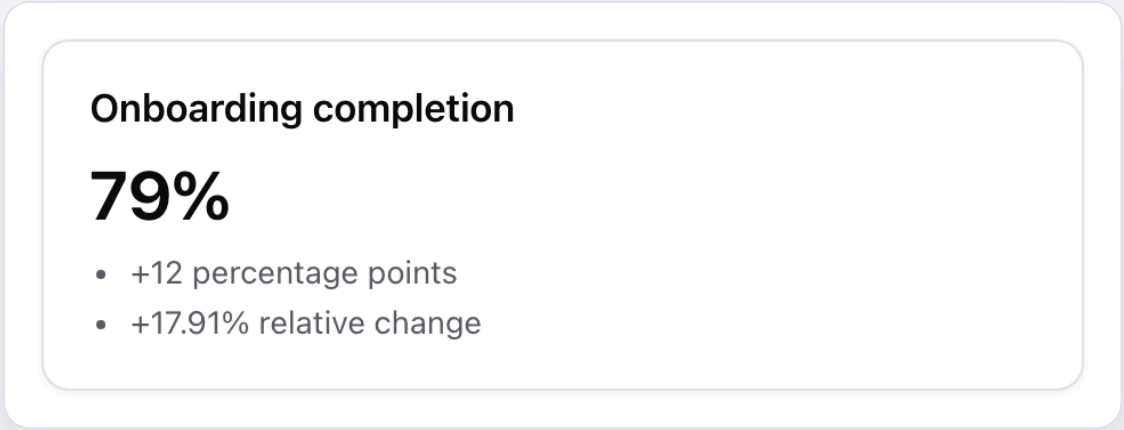
Wrong digits in most runs, usually with no review note; once it claimed it had “copied exactly”. A stricter prompt helped one time in three.

So the page, not the model, lists every figure it drafted, and Create waits until the person has checked them.



METRIC CARDS

The one place the tool does arithmetic



67% → 79% is +12 percentage points *and* +17.91% relative change. Both shown, each named.

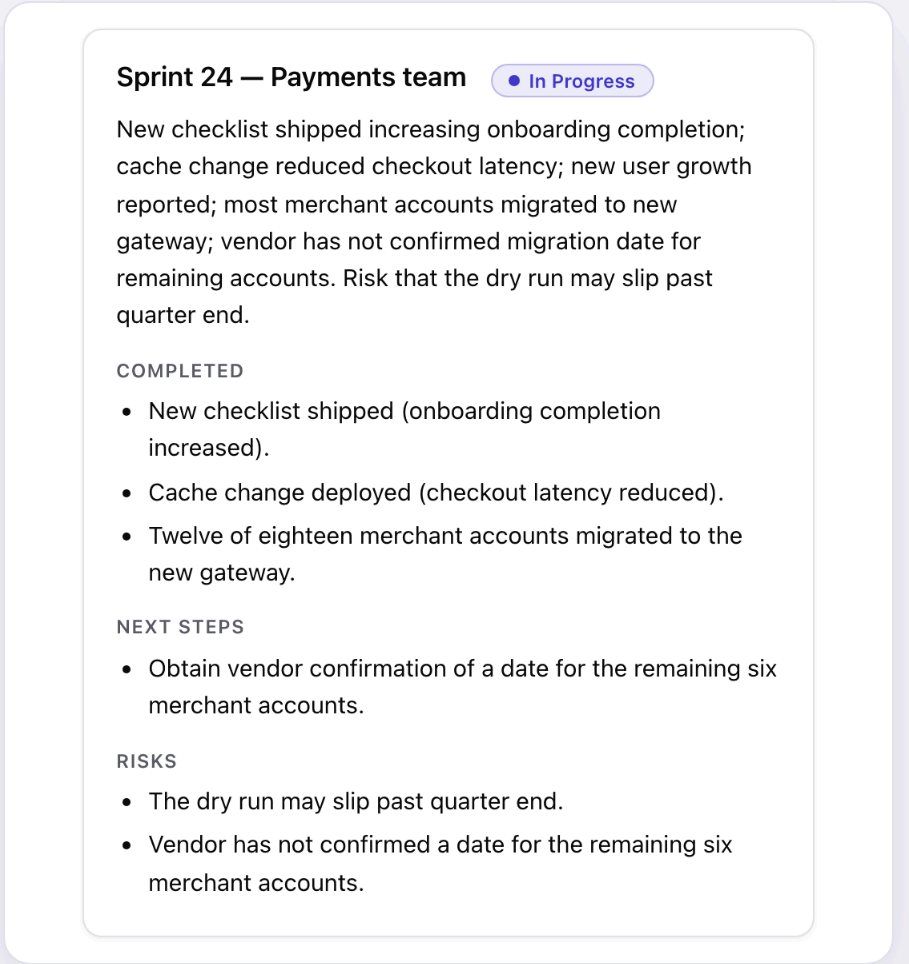
A pure function on the server, covered by an 11-row table: zero baselines, negatives, decimals, non-finite input.

SHARING

One render, five ways out

FORMAT	WHERE IT GOES
Plain text	Chat, any client
Rich text	Email, inline styles only
PNG	One card, editing controls removed
HTML file	The whole report, self-contained
PDF	Browser print, with a print stylesheet

Every export is assembled from what the server already rendered, so it cannot differ from the preview.



CHAPTER 4

Architecture

A thin browser, one server, one model, deployed by a script you can read in one sitting.



↓ 2 slides below

SYSTEM

Three parts, one direction of trust

BROWSER

Editor and state

ES modules, no build step · IndexedDB for drafts and images · images downscaled before they are sent

→

FASTAPI

Validate and render

Pydantic v2 card models · Jinja2 renders preview, email and text · one error envelope

→

AZURE OPENAI

Draft only

gpt-5-mini, text and image input · strict JSON schema · behind a provider interface

Docker, non-root uid 10001, one worker

Azure Container Apps, one replica, scales to zero

Key held as a secret reference

DECISIONS

Choices I would defend in review

DECISION	WHY
The server renders every preview	One source of truth for screen, email and text, so exports cannot drift
No frontend build step	Fewer moving parts; the UI kit (Basecoat) ships precompiled, so the rule held
Images leave the browser only on request	Nothing stored or retained; the one exception is shown, opt-in and downscaled
Reasoning effort: low	Measured on one note: minimal 2.3 s, low 4.4 s, default 11.6 s, same facts. A setting, not a constant
Scale to zero, one replica	Measured cold start 2.3 s; the rate limiter lives in one process

CHAPTER 5

Security

Public, no login, and every AI call costs money. So the question is what an attacker would try, not what the tool does.



↓ **3 slides** below

THREAT MODEL

What an attacker would try, and what stops it

ATTACK	CONTROL
An endless request body	Counted as it arrives; 413 at 64 KiB (1.5 MiB with an image)
One visitor using up the AI for everyone	Per client: 6 a minute, 1 in flight
Many addresses running up the bill	App ceiling: 30 a minute. Azure deployment capped at 10K tokens a minute
A spend spike nobody notices	Azure alert emails me over 200K tokens in an hour
A forged X-Forwarded-For	Only the address the Azure ingress appended counts
Script injection	Autoescaping, text-only insertion, CSP `script-src 'self'`
Prompt injection, in text or an image	Rules and content sent apart, strict schema, no tools, human review
A stolen key	Secret reference; never in the image, the page or an error

COST CONTROLS ON AZURE

Layers between an attacker and the bill

LAYER	WHAT IT DOES	STATUS
App rate limit	6 a minute per visitor, 30 in total	**Done**
Azure deployment cap	10K tokens a minute, down from 100K; holds even if the app is bypassed	**Done**
Cap reached	Shown as “busy, try again in a minute”, not “unreachable”	**Done**
Token-usage alert	Email when usage passes 200K tokens in an hour	**Done**
Subscription spending limit	Credit exhausted stops resources; no card is charged	**On**
Budget alert	Monthly budget with email at 50, 80 and 100%	Next
Front Door WAF	Per-IP rate rules at the edge; from about $35 a month	Production

WHAT THE REVIEW FOUND

Four real problems, each fixed with a test that fails without it

FOUND	EVIDENCE AND FIX
A 10 MiB body with no length was read whole	Raw-socket test on a real server; now refused at the limit
Starlette 0.41.3: eight advisories, one reachable	CVE-2025-62727 via `/static`; upgraded, pip-audit on every push and weekly
No security headers on the live site	CSP and six more on every response; every browser test fails on a CSP violation
One shared AI rate limit	Per client; seven forged addresses tried live, refused from the sixth

Azure-side cost controls are on the previous slide. Left for production: login and a WAF.

Engineering excellence

Four practices, each with evidence you can open in the repository.

DOCUMENTATION

4 documents

96 references from code

CONFIGURATION

0 hard-coded

model settings, secrets as references

TESTING

251 tests

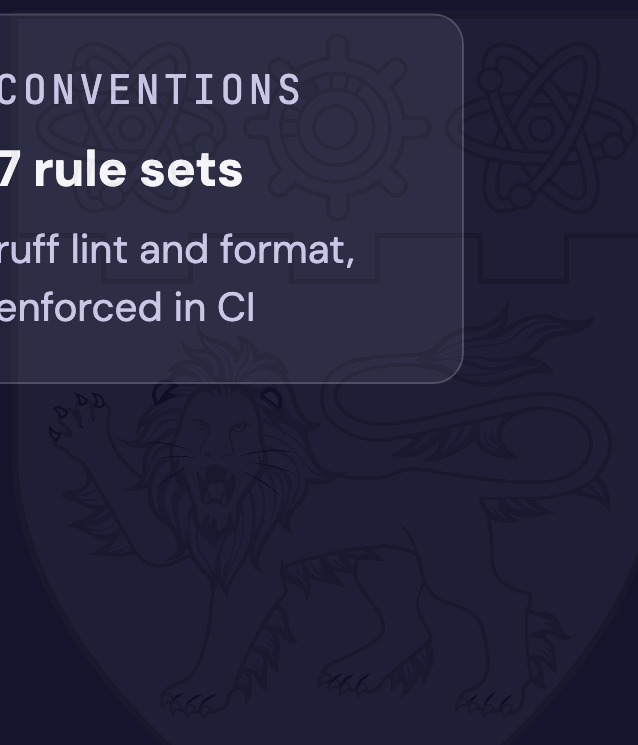
13 security tests, 11 live checks

CONVENTIONS

7 rule sets

ruff lint and format, enforced in CI

↓ **5 slides** below



DOCUMENTATION

Four documents, each with one job

SCOPE.MD

What, and what not

Card types, AI limits, sharing rules, out of scope

ARCHITECTURE.MD

How

14 sections: models, API, state, persistence, security

TEST_PLAN.MD

What to prove

12 sections, written before the code

TEST-REPORT.MD

What was proven

Dated; a check not run is never recorded as passed

- Code cites the rule it implements: **96** section references such as `docs/architecture.md §7`
- README splits *What works today* from *Planned, not built*
- Before submitting, every document was checked against the code: a stale file tree, a feature described but never built, and two mis-cited sections were fixed

CONFIGURATION

Nothing about the model is hard-coded

```
class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=".env", extra="ignore")

    ai_enabled: bool = False
    azure_openai_api_version: str = "2025-04-01-preview"
    # On a reasoning model this budget covers reasoning
    # tokens as well as the answer: measured against
    # gpt-5-mini, 900 was spent entirely on reasoning
    # and returned an empty, incomplete response.
    ai_max_output_tokens: int = 2_000
    # "minimal" 2.3s, "low" 4.4s, model default 11.6s.
    # All three extracted the same facts, so this
    # defaults to the cheap end.
    ai_reasoning_effort: str = "low"
```

- One `Settings` class, read once; `.env.example` documents every variable
- Safe default: AI off, the app still works, the AI endpoint returns 503
- Each default carries the measurement behind it
- The key is a Container Apps secret reference; a check fails if it is a literal or appears in the page
- Dependencies locked; vendored browser files listed with version, source, licence and sha256

TESTING

251 tests, and proof that they bite

89

unit

Metric table, card models, AI and image schemas

75

integration

API, templates, errors, 13 security tests

87

browser

Workflows and axe scans; each one also a CSP check

11

live checks

verify.sh against the deployed URL

- A fake provider: the suite needs no credentials and spends nothing; the real model is covered by live cases and the deployment check
- Fixes are proven to bite: remove the guard and the test goes red (export guard, figure check, forged-header limit, blank-PNG check)
- CI runs four jobs on every push and weekly: lint and tests · browser and accessibility · dependency audit · Docker build with header and non-root checks

Rules a machine enforces, reasons a person can read

```toml
[tool.ruff]
line-length = 100
target-version = "py313"

[tool.ruff.lint]
select = ["E", "F", "I", "UP", "B", "SIM", "S"]

[tool.ruff.lint.per-file-ignores]
# assert is the point of a test
"tests/*" = ["S101"]
# Depends() in a default is how FastAPI
# declares dependencies
"app/main.py" = ["B008"]
```

- Ruff lint and format in CI, including the `S` security rules; every exception states its reason
- Pydantic discriminated union: a card is progress, metric or image; unknown fields are rejected
- One error envelope, `{error: {code, message, fields}}`, never echoing input
- Model output reaches the page through `textContent`, never `innerHTML`
- Commit messages explain why, not only what

WHAT THE TESTS CAUGHT

Four failures, and what each one changed

WHAT BROKE	HOW IT WAS FOUND	WHAT CHANGED
PNG exports were blank	Size assertions passed; opening the file did not	A bytes-per-pixel check that fails if the bug returns
A 900-token budget returned nothing	Live call: reasoning used the whole budget	Budget 2,000; an incomplete answer is rejected
Small print misread: 2466 as 2496	Live model, with no warning	The page lists every figure; Create waits for a check
Hover text at 4.2:1 contrast	CI only; local runs passed	Hover colours fixed; hover scans in both themes

Also this week: an animation exposed “Saved locally” shown 400 ms before it was true, and CI caught the framework upgrade silently dropping HEAD support.

HONEST LIMITS

What is not proven yet, and what comes next

NOT PROVEN YET

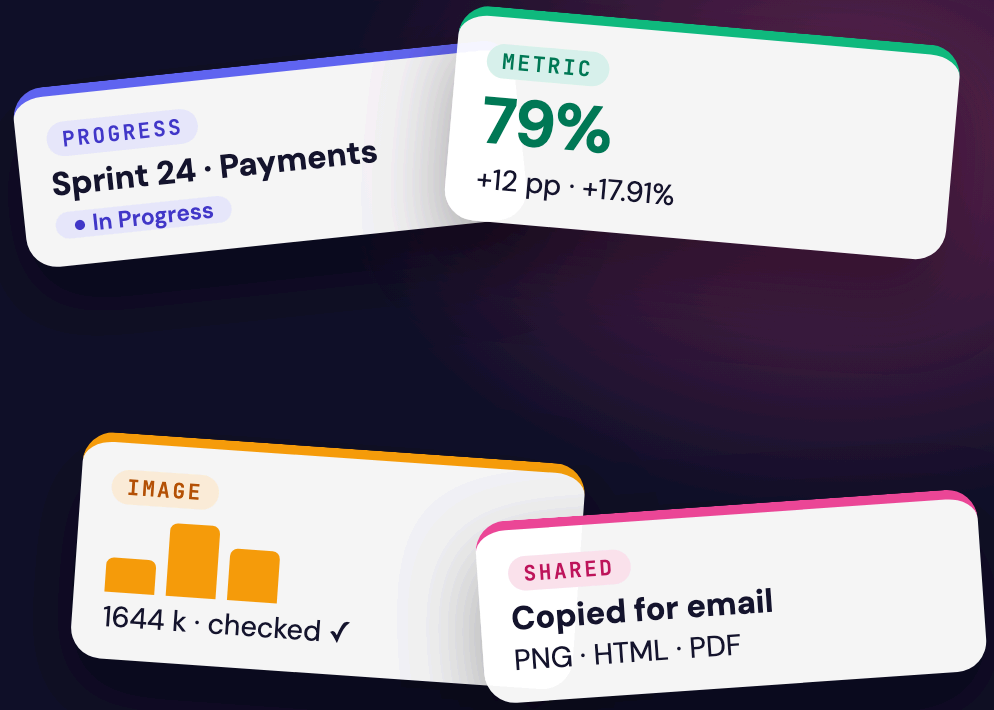
- A small live sample, text and images: not an accuracy claim
- No screen-reader session; small screens and 200% zoom untested
- No login or WAF, and no budget alert yet: a public POC

NEXT

- An evaluation set, text and images, with expected output per case
- Front Door WAF, a budget alert, managed identity instead of a key
- Client checks in Outlook, Gmail, Slack and Teams

Thank you

251 tests · **11** live checks · **4** security fixes · **1** misread caught live



Zhang Dingpu · NTU Singapore · College of Computing and Data Science

Live · ca-reporting-builder.kindbush-e2227a04.koreacentral.azurecontainerapps.io

Code · github.com/maxwellzhang-dev/project-reporting-builder